Software Engineering Models and Methods

TLA$^+$: [2/4] Abstractions and Refinements

assoc. prof. Karolis Petrauskas

Vilnius University
Department of Software Engineering
Institute of Computer Science

2026

Русский военный корабль

0LjQtNC4INC90LDRhdGD0Lk=

Outline

Introduction

On Recursion

On Composition

Refinements
- Example: Connection Locks
- Example: Paxos

Discussion and Follow-up

References

This lecture is on choosing right abstractions.

- Too detailed spec is complex and hard to check.
  Either by model checking or proving.
- Too abstract spec can omit important details.
  But they are easy to understand, check and prove.
- We can combine abstract and detailed specs in a similar way, as we combine components via their interfaces when building systems.
- The spec is Math, not a code.
  This allows to have more freedom for expressing various things, including the composition.

Outline

Introduction

On Recursion

On Composition

Refinements
- Example: Connection Locks
- Example: Paxos

Discussion and Follow-up

References

On Recursion

It is usually better to avoid recursive algorithms in TLA$^+$.

- The TLA$^+$ language was designed to model distributed systems.
- Distributed systems are usually complex because no reliable network exist.
- So we emphasize communication logic and leave in-process logic as abstract as possible.

On Recursion (cont.)

For example, consider a function *FirstIndexOf* that returns first occurrence of an element in a list or an array.

```
MODULE TLA_Lecture_Recursion
EXTENDS Sequences, Naturals

Recursive variant of FindFirst in a list (tuple).

RECURSIVE FindFirstRec(_, _, _)
FindFirstRec(e, l, n) ≜
  IF l = ⟨⟩
   THEN 0
   ELSE IF Head(l) = e
           THEN n
           ELSE FindFirstRec(e, Tail(l), n + 1)
FindFirst(e, l) ≜ FindFirstRec(e, l, 1)
```

- We use a tuple to model a list. It is 1-based.
- 0 stands for "Not found".
- *Head* and *Tail* are defined in the *Sequences* module.
- The algorithm is in the usual functional style.

On Recursion (cont.)

The above variant tells, how to find the first occurrence of the value (operational semantics). That's usually not important in the context of more high-level specification.

Declarative version of the *FindFirst*, using CHOOSE .

$$
\begin{array}{ll}
FindFirstCh(e, l) \triangleq & \\
\quad \text{CHOOSE } n \in Nat : & \\
\quad\quad \text{IF } \exists\, i \in \text{DOMAIN } l : l[i] = e & \\
\quad\quad\quad \text{THEN } \wedge\, n \in \text{DOMAIN } l & \text{Result must be well defined.} \\
\quad\quad\quad\quad\quad \wedge\, l[n] = e & \text{The element is found.} \\
\quad\quad\quad\quad\quad \wedge\, \forall\, i \in 1 \,..\, n-1 : l[i] \neq e & \text{It is the first occurance.} \\
\quad\quad\quad \text{ELSE } n = 0 &
\end{array}
$$

- In this case we don't specify the algorithm, instead we just specify properties the result must satisfy (denotational semantics).
- It is safe to use CHOOSE here, because the solution is always unique.
- It is important to consider only the first occurrence, $\exists$ would pick any of them.

On Recursion (cont.)

If we want to avoid using CHOOSE operator, we can use $\exists$ instead.

Declarative bersion of the *FindFirst* using $\exists$.

```
FindFirstEx(e, l, P(_)) ≜
  ∨ ∃ n ∈ DOMAIN l :
      ∧ l[n] = e
      ∧ ∀ i ∈ 1 .. n − 1 : l[i] ≠ e
      ∧ P(n)
  ∨ ∀ n ∈ DOMAIN l :
      ∧ l[n] ≠ e
      ∧ P(0)
```

- $\exists$ results in a Boolean value, not an index we need.
- We can pass an operator $P(_)$ to be called with the result.
- Negative case must have the negative guard, because in math there is no short-circuit rule.

On Recursion (cont.)

We should test our operators, to be sure they behave as expected.

Test cases.

$$
\begin{array}{l}
\text{VARIABLE } tc \\
Init \triangleq \land\ tc \in \{\langle 2, 3, \langle 5, 3, 2, 7, 2\rangle\rangle, \\
\qquad\qquad\qquad \langle 7, 4, \langle 5, 3, 2, 7, 2\rangle\rangle, \\
\qquad\qquad\qquad \langle 1, 0, \langle 5, 3, 2, 7, 2\rangle\rangle\} \\
Next \triangleq \text{UNCHANGED } \langle tc\rangle \\
Spec \triangleq Init \land \Box[Next]_{\langle tc\rangle} \\
\\
TestRec \triangleq tc[2] = FindFirst(tc[1], tc[3]) \\
TestCh \triangleq tc[2] = FindFirstCh(tc[1], tc[3]) \\
TestEx \triangleq FindFirstEx(tc[1], tc[3], \text{LAMBDA } rez : rez = tc[2]) \\
\text{THEOREM } Spec \Rightarrow \Box(TestRec \land TestCh \land TestEx)
\end{array}
$$

- *Init* defines all the test cases we want to check.
- The layout of the test case is the same as we saw in the previous lecture.
- We can use LAMBDA expressions in TLA$^+$ as well. It is passed to stand for $P(_)$.

On Recursion (cont.)

It is usually no need to define such operators as *FindFirst*. We can just use the needed properties inline, usually in the $\exists$ expression.

It is usually more convenient to use $\exists$ expressions inline.

```
57 TestInline ≜
58   LET l ≜ tc[3]
59       e ≜ tc[1]
60       r ≜ tc[2]
61   IN  ∨ ∃ n ∈ DOMAIN l :
62           ∧ l[n] = e
63           ∧ ∀ i ∈ 1 .. n − 1 : l[i] ≠ e
64           ∧ TRUE                                  Do our actual stuff here.
65       ∨ ∀ n ∈ DOMAIN l : l[n] ≠ e ∧ r = 0         Usually not needed, just for TC.
66 THEOREM Spec ⇒ □TestInline
```

- We assume here the spec needs the $\exists$ expression on line 61.
- TRUE on line 64 is a placeholder for all the user logic.
  TRUE is a convenient placeholder for various incomplete places.
- The *FirstIndexOf* is modelled just by lines 62-63.
- Other lines are just to put it all to the test case.

On Recursion (cont.)

It is often more convenient to define predicates checking some property instead of an operator satisfying it.

It is convenient to define predicates for further reuse. The same approach was used for defining different network partitions.

```
72  IsFirst(e, l, n) ≜
73    ∧ n ∈ DOMAIN l
74    ∧ e = l[n]
75    ∧ ∀ i ∈ 1 .. n − 1 : l[i] ≠ e
76  TestInlinePred ≜
77    LET l ≜ tc[3]
78        e ≜ tc[1]
79        r ≜ tc[2]
80    IN  ∨ ∃ n ∈ DOMAIN l :
81            ∧ IsFirst(e, l, n)
82            ∧ TRUE                                  Do our actual stuff here.
83        ∨ ∀ n ∈ DOMAIN l : l[n] ≠ e ∧ r = 0         Usually not needed, just for TC.
84  THEOREM Spec ⇒ □TestInlinePred
```

- Here we defined a predicate *IsFirst*(e, l, n).
- We then use it as shown on line 81.

Outline

Introduction

On Recursion

On Composition

Refinements
- Example: Connection Locks
- Example: Paxos

Discussion and Follow-up

References

On Composition

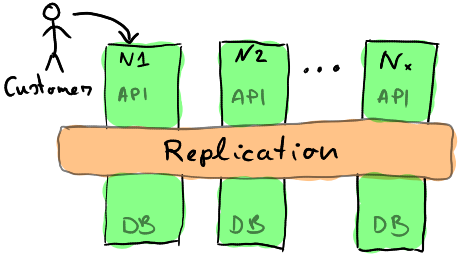
Compose using interfaces, not implementations, if possible.

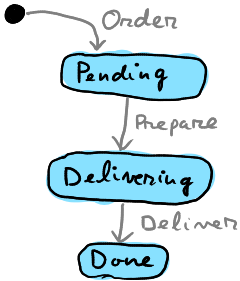
- In programming we need a particular component implementing an interface we depend upon in order to have a runnable system.
- In math, we don't need to copy theorem proofs to have our problem solved or proved. We just depend on properties proven by the theorem. E.g. by the Pythagorean theorem.
- Implementation usually has its own state space. If we include it to our spec, the state space increases rapidly.

Use **operational** semantics to describe the algorithm you investigate and **denotational** semantics for the components you use. Provide denotational interface for your algorithm as well.

Example: Order Management System (OMS)

- A distributed system consisting of x nodes.
- Each node has its own local DB and provides an API.
- Customer can access any node any time.
- Replication solution is used to keep all nodes in sync.
- We have solutions for the DB and replication.
- We want to specify the order management, but it depends on the replication and the storage.





Example OMS: Spec for the Local DB

- Databases are complicated, because they must ensure high concurrency and performance at the same time.
- But we are not designing the DB, we just use it.
- We define a spec reflecting guarantees provided by the DB:
  - Its a key-value DB.
  - It provides linearizability.
  - It provides persistence.

MODULE *TLA_Lecture_Composition_LocalDB*

This *DB* provides a key-value interface with a set semantics.

$$\text{CONSTANTS } Keys,\ Values,\ NoVal$$
$$New \triangleq \langle\rangle$$
$$Get(k, db) \triangleq \text{IF } k \in \text{DOMAIN } db \text{ THEN } db[k] \text{ ELSE } NoVal$$
$$Set(k, v, db) \triangleq [kk \in \text{DOMAIN } db \cup \{k\} \mapsto \text{IF } kk = k \text{ THEN } v \text{ ELSE } db[kk]]$$
$$TypeOK(db) \triangleq \exists K \in \text{SUBSET } Keys : db \in [K \to Values]$$

NOTE: That's an interface provided (implemented) by the DB.

Example OMS: Spec for the Replicator

- Replication algorithms are complex as well, because they have to cope with unreliable network and guarantee the consistency.
- But we are going to use an existing solution, so we specify its properties (not very realistic, for simplicity of the example).
  - It works in a synchronous manner.
  - It ensures that all the nodes will receive the same value.
  - The data is replicated instantly and atomically.

```
MODULE TLA_Lecture_Composition_Replicator
Writes the same values to all the specified nodes synchronously.
LOCAL INSTANCE Sequences
CONSTANTS Nodes, W(_, _, _)
VARIABLE backends
TypeOK ≜ DOMAIN backends = Nodes
Init ≜ TypeOK
Write(n, k, v) ≜ backends' = [nn ∈ Nodes ↦ W(k, v, backends[nn])]
```

NOTE: Again, that's only an interface provided by the replicator.

Example OMS: Spec of the system in question

- In the modelled system we have the customers and their orders.
- From the topology point of view, we have several nodes, and local DBs on each of them.

```
MODULE TLA_Lecture_Composition
CONSTANTS Nodes, Customers, Orders
VARIABLE ldbs  Local Databases
vars ≜ ⟨ldbs⟩
States ≜ {"pending", "delivering", "done"}
OrderStates ≜ Orders × States
NoOrder ≜ ⟨⟩
```

- Orders can be in the states: *pending*, *delivering* and *done*.
- The system stores the orders along with their states.
- It is safe to use empty tuple for *NoOrder*, because valid *OrderStates* are always 2-tuples.

Example OMS: Spec of the system in question (cont.)

Next we are going to use the previously defined modules for the *LocalDB* and the *Replicator*.

```
LocalDB ≜ INSTANCE TLA_Lecture_Composition_LocalDB
            WITH Keys ← Customers,
                 Values ← OrderStates,
                 NoVal ← NoOrder
Replicator ≜ INSTANCE TLA_Lecture_Composition_Replicator
               WITH backends ← ldbs,
                    W ← LAMBDA k, v, db : LocalDB!Set(k, v, db)
```

- We use instance of that spec with renaming. I.e. *New* in the *LocalDB* spec will become *LocalDB*!*New* in this spec.
- We map all the constants and variables for the instantiated modules.
- We can have several different instances of the same module under different names.
- Consider, how a Lambda expression is passed for the *W* operator. It's a DB write function, that was abstracted in the *Replicator* spec.

Example OMS: Spec of the system in question (cont.)

Here we have the "business specific" actions of the system. We assume here that writes to the DB are performed via replication, and reads are done locally.

$$
\begin{array}{l}
Order \triangleq \exists\, c \in Customers,\ n \in Nodes,\ o \in Orders : \\
\quad \text{LET } state \triangleq LocalDB!Get(c,\ ldbs[n]) \quad \text{Read from Local } DB \\
\quad \text{IN } \quad \land\ state = NoOrder \\
\qquad\quad\ \land\ Replicator!Write(n,\ c,\ \langle o,\ \text{"pending"}\rangle) \quad \text{Write replicated.} \\
Prepare \triangleq \exists\, c \in Customers,\ n \in Nodes : \\
\quad \text{LET } state \triangleq LocalDB!Get(c,\ ldbs[n]) \\
\quad \text{IN } \quad \land\ state \neq NoOrder \land state[2] = \text{"pending"} \\
\qquad\quad\ \land\ Replicator!Write(n,\ c,\ \langle state[1],\ \text{"delivering"}\rangle) \\
Deliver \triangleq \exists\, c \in Customers,\ n \in Nodes : \\
\quad \text{LET } state \triangleq LocalDB!Get(c,\ ldbs[n]) \\
\quad \text{IN } \quad \land\ state \neq NoOrder \land state[2] = \text{"delivering"} \\
\qquad\quad\ \land\ Replicator!Write(n,\ c,\ \langle state[1],\ \text{"done"}\rangle)
\end{array}
$$

NOTE: We are not calling the *Set* for the databases directly here. That's done via the Lambda passed to the *Replicator* at the instantiation time.

Example OMS: Spec of the system in question (cont.)

Here we have the spec.

$$
\begin{aligned}
Init &\triangleq \land\ ldbs = [n \in Nodes \mapsto LocalDB!New] \\
&\quad\ \land\ Replicator!Init \\
Next &\triangleq Order \lor Prepare \lor Deliver \\
Spec &\triangleq Init \land \Box[Next]_{vars} \land \mathrm{WF}_{vars}(Next)
\end{aligned}
$$

We can't have *LocalDB*!*Init*, unless we have separate variables for all the DB instances (variable assignment vs function update).

$$
\begin{aligned}
TypeOK &\triangleq \land\ \text{DOMAIN}\ ldbs = Nodes \land \forall\, n \in Nodes : LocalDB!TypeOK(ldbs[n]) \\
&\quad\ \land\ Replicator!TypeOK \\
Consistent &\triangleq \forall\, n1,\, n2 \in Nodes : ldbs[n1] = ldbs[n2] \\
WillBeNotEmpty &\triangleq \Diamond\exists\, n \in Nodes : ldbs[n] \neq LocalDB!New \\
\text{THEOREM}\ Spec &\Rightarrow \Box(TypeOK \land Consistent) \land WillBeNotEmpty
\end{aligned}
$$

And here we are reusing the *LocalDB*!*TypeOK* to validate the state of the DB. State of the DB is opaque to this module.

On Composition, again

We have looked at composition by instantiating other modules.

- Very often it is better to avoid such splitting of the spec to modules; just specify everything in one spec.
- The example above would be simpler to understand without this decomposition.
- Sometimes refinements justify this complexity.

TLA$^+$ has other means for specification composition as well:

- Using the EXTENDS keyword.
- By co-joining several specifications and making their behaviours interleaving.

Outline

Introduction

On Recursion

On Composition

Refinements
- Example: Connection Locks
- Example: Paxos

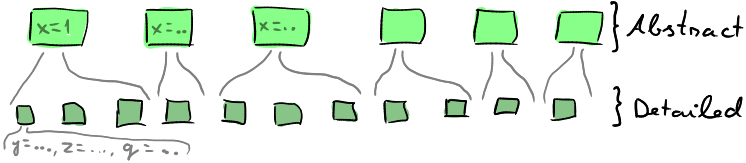
Discussion and Follow-up

References

Refinements

Lamport (2019)

- Refinements allow to have specification at multiple abstraction levels.
- More detailed specification implements the abstract one. Often the abstract specification is declarative (requirements) and the detailed one – imperative (particular algorithm implementing requirements).
- In TLA$^+$ the refinement is expressed as an implication.
- Usually the refining spec has more variables, more states, more actions. By refinement mapping we map refining states to the abstract ones. This way, each refining behaviour is also a behaviour in the abstract spec.



Outline

Introduction

On Recursion

On Composition

Refinements
- Example: Connection Locks
- Example: Paxos

Discussion and Follow-up

References

Example: Connection Locks

We can define a specification that is very abstract and easy to understand. Its purpose is to specify the observable behaviour.

```
MODULE TLA_Lecture_ConnLock_v0
That's the most abstract version of this problem. It should express only the core behaviour,
without any implementation. We will use this spec later to show that a specific implementation
actually refined this spec.

CONSTANT SID                    Abstract session IDs.
VARIABLE sstate                 Abstract session states.
Init  ≜  sstate = [s ∈ SID ↦ FALSE]
Next  ≜  ∃ s ∈ SID : ∧ ¬sstate[s]
                     ∧ sstate′ = [sstate EXCEPT ![s] = TRUE]
Spec  ≜  Init ∧ □[Next]_sstate ∧ WF_sstate(Next)
TypeOK  ≜  sstate ∈ [SID → BOOLEAN ]    Type correctness.
WBConn  ≜  ◇□∀ s ∈ SID : sstate[s]      Will Be Connected.
THEOREM Spec ⇒ □TypeOK ∧ WBConn
```

It is declarative, because we provide no implementation details, how the sessions are initialized. We just say, that they are initialized one by one.

Example: Connection Locks (cont.)

We have defined the algorithm for the connection pool initialization in the previous lecture.

```
---------------------- MODULE TLA_Lecture_ConnLock_v1 ----------------------
\* ...
---------------------------------------------------------------------------
A == INSTANCE TLA_Lecture_ConnLock_v0
         WITH SID    <- S,
              sstate <- sessions
THEOREM Spec => A!Spec
===========================================================================
```

- We define A to be an instance of the abstract specification.
- We use S to instantiate SID constant in the abstract spec.
- The mapping for its variables is also trivial in this case.

We can check refinement using TLC by specifying $A!Spec$ as a temporal property.

Example: Connection Locks (cont.)

Assuming $S = \{s_0\}$ with an initial state

```
/\ initLock = Free
/\ connLock = Free
/\ initProc = [s_0 !-> "wait"]      ===>      sstate = [s_0 !-> FALSE]
/\ callProc = [s_0 !-> "wait"]
/\ sessions = [s_0 !-> FALSE]
```

after $OnCallConn(s_0)$ it becomes

```
/\ initLock = Free
/\ connLock = s_0
/\ initProc = [s_0 !-> "wait"]      ===>      sstate = [s_0 !-> FALSE]
/\ callProc = [s_0 !-> "conn"]
/\ sessions = [s_0 !-> FALSE]
```

and after $OnCallDone(s_0)$:

```
/\ initLock = Free
/\ connLock = Free
/\ initProc = [s_0 !-> "wait"]      ===>      sstate = [s_0 !-> TRUE]
/\ callProc = [s_0 !-> "done"]
/\ sessions = [s_0 !-> TRUE]
```

So 3 steps are mapped to 1 in this scenario. It would be more steps in the case of *OnInit* actions.

Example: Connection Locks (cont.)

Some questions for a discussion:

1. How refinement is similar to implementation of an interface in some OOP language?
2. What is mapped in the refinement mapping?
3. Why is it useful to have refinements?
4. If you prove some property for an abstract specification, does it hold for the refining specification?
5. Does it apply in the opposite direction?
6. How action atomicity is leveraged at various levels of abstractions?

Outline

Introduction

On Recursion

On Composition

Refinements

- Example: Connection Locks
- Example: Paxos

Discussion and Follow-up

References

Example: Paxos

Lamport (2001)

$$\Pi\alpha\xi o\varsigma$$

We will consider the Paxos algorithm as

1. a non-trivial example of a refinement as well as
2. a realistic specification for a distributed systems algorithm.

Paxos is an algorithm for reaching consensus in a distributed asynchronous system, where messages can be duplicated, lost or reordered.

- The part-time parliament, Lamport (1998);
- Paxos Made Simple, Lamport (2001).

Example: Paxos (cont.)

Lamport (2001)

There are multiple variants of this algorithm, like multi-paxos, fast paxos, cheap paxos, flexible paxos, egalitarian paxos, generalized paxos, and other.

Here we will consider only the the classic / single synode algorithm, based on the "Paxos Made Simple"[1] paper.

Abstract:

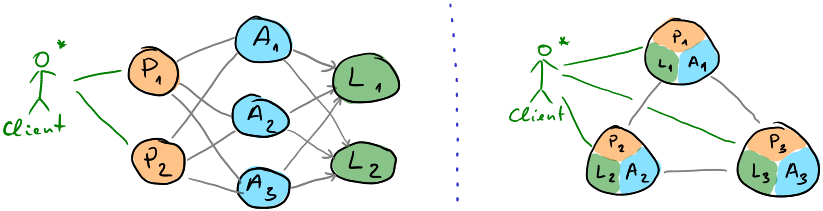
The Paxos algorithm, when presented in plain English, is very simple.

[1] There is a mistake in the paper, the accompanying spec is correct.

Paxos — Participants

The algorithm is formulated for a network of participants of types:

- Proposers — are proposing a value (handles an user input).
- Acceptors — vote for the proposed values.
- Learners — learns the decided value (handles a decision result).



Paxos — Guarantees

The following guarantees are provided by the algorithm:

1. Only a value that has been proposed may be chosen.
2. Only a single value is chosen.
3. A process never learns that a value has been chosen unless it actually has been.

NOTE: A progress is only guaranteed in the case, when values are proposed by single proposer (a leader). Multiple concurrent proposers can cause live-lock (this can also be solved using randomized delays).

Paxos — Abstract Specification

The following specification (excerpt from `Consensus.tla`) is a trivial abstraction of the consensus algorithm:

```
--------------- MODULE Consensus ----------------------
CONSTANTS Acceptors, Values
VARIABLE chosen
Init == chosen = {}
Next == /\ chosen = {}
        /\ \E v \in Values : chosen' = chosen \cup {v}
Spec == Init /\ [][Next]_chosen
=====================================================
```

The Paxos algorithm is formulated as a refinement of the above spec (excerpt from `Paxos.tla`):

```
-----------------------------------------------------
chosenBar == {v \in Values : Chosen(v)}
C == INSTANCE Consensus WITH chosen <- chosenBar
THEOREM Refinement == Spec => C!Spec
-----------------------------------------------------
```

Here, if a behaviour satisfies *Spec*, then it also satisfies *C!Spec*. The *chosenBar* stands for the refinement mapping.

Paxos — Made Simple

Phase 1. (a) A proposer selects a proposal number n and sends a *prepare* request with number n to a majority of acceptors.

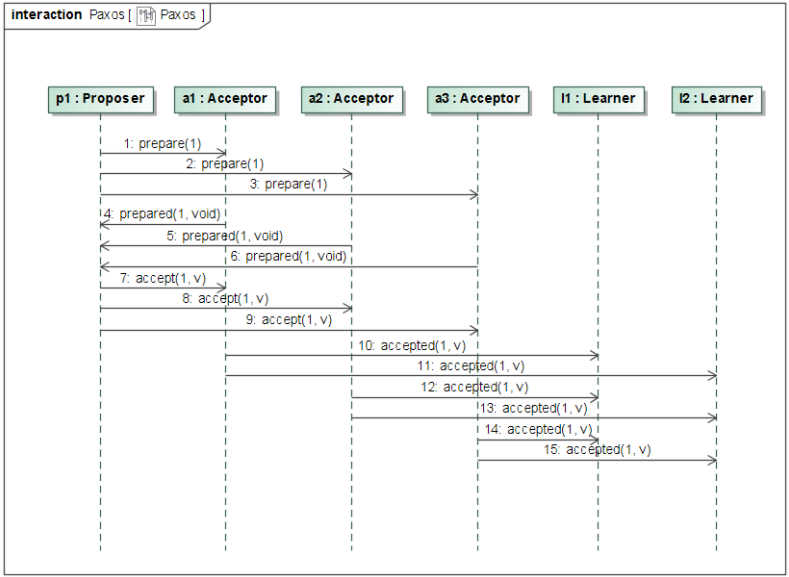
(b) If an acceptor receives a *prepare* request with number n greater than that of any *prepare* request to which it has already responded, then it responds to the request with a promise not to accept any more proposals numbered less than n and with the highest-numbered proposal (if any) that it has accepted.

Phase 2. (a) If the proposer receives a response to its *prepare* requests (numbered n) from a majority of acceptors, then it sends an *accept* request to each of those acceptors for a proposal numbered n with a value v, where v is the value of the highest-numbered proposal among the responses, or is any value if the responses reported no proposals.

(b) If an acceptor receives an *accept* request for a proposal numbered n, it accepts the proposal unless it has already responded to a *prepare* request having a number greater than n.

To learn that a value has been chosen, a learner must find out that a proposal has been accepted by a majority of acceptors. The obvious algorithm is to have each acceptor, whenever it accepts a proposal, respond to all learners, sending them the proposal. This allows learners to find out about

Paxos — Basic Message Exchange



Paxos — TLA$^+$ Specification

Lamport (2001)

The complete formal specification of the algorithm along with the proofs for its correctness is provided in the following files:

- specs/Paxos/Consensus.pdf
- specs/Paxos/Paxos.pdf

This specification can be used as a good example of how to abstract unnecessary parts out, e.g.:

- How message passing is modelled, covering lost, duplicated and delayed messages.
- How quorums are used instead of majorities, making the entire spec more general.
- How state is minimized by expressing properties based on messages instead of state variables.
- and many more.

Also consider proofs as an example, how to approach them.

Paxos — TLA$^+$ Specification (cont.)

Lamport (2001)

```
CONSTANTS Acceptors, Values, Quorums

ASSUME QuorumAssumption ≜
          ∧ Quorums ⊆ SUBSET Acceptors
          ∧ ∀ Q1, Q2 ∈ Quorums : Q1 ∩ Q2 ≠ {}

The following lemma is an immediate consequence of the assumption.
LEMMA QuorumNonEmpty ≜ ∀ Q ∈ Quorums : Q ≠ {}
BY QuorumAssumption

Ballots ≜ Nat

VARIABLES msgs,      The set of messages that have been sent.
          maxBal,    maxBal[a] is the highest-number ballot acceptor a
                      has participated in.
          maxVBal,   maxVBal[a] is the highest ballot in which a has
          maxVal      voted, and maxVal[a] is the value it voted for
                      in that ballot.
```

Constants as sets + assumptions.

Ballots is an alias for *Nat*, this allows overriding it in the model to make it finite.

Paxos — TLA$^+$ Specification (cont.)

Lamport (2001)

$$vars \triangleq \langle msgs, maxBal, maxVBal, maxVal \rangle$$

$$Send(m) \triangleq msgs' = msgs \cup \{m\}$$

$$None \triangleq \text{CHOOSE } v : v \notin Values$$

$$\text{LEMMA } NoneNotAValue \triangleq None \notin Values$$
$$\text{BY } NoSetContainsEverything \text{ DEF } None$$

$$\begin{aligned} Init \triangleq\ & \land msgs = \{\} \\ & \land maxVBal = [a \in Acceptors \mapsto -1] \\ & \land maxBal\ \ = [a \in Acceptors \mapsto -1] \\ & \land maxVal\ \ = [a \in Acceptors \mapsto None] \end{aligned}$$

Blackboard approach to the network.
Consider definition of *None*.
Variables are functions of nodes.

Paxos — TLA$^+$ Specification (cont.)

Lamport (2001)

$$
\begin{aligned}
Next &\triangleq \lor \exists\, b \in Ballots : Phase1a(b) \lor Phase2a(b) \\
& \lor \exists\, a \in Acceptors : Phase1b(a) \lor Phase2b(a) \\
Spec &\triangleq Init \land \Box[Next]_{vars}
\end{aligned}
$$

The overall spec formula is obvious.

Paxos — TLA$^+$ Specification (cont.)

Lamport (2001)

$$
\begin{array}{lll}
Messages & \triangleq & [type : \{\text{“1a”}\},\ bal : Ballots] \\
 & \cup & [type : \{\text{“1b”}\},\ bal : Ballots,\ maxVBal : Ballots \cup \{-1\}, \\
 & & \quad maxVal : Values \cup \{None\},\ acc : Acceptors] \\
 & \cup & [type : \{\text{“2a”}\},\ bal : Ballots,\ val : Values] \\
 & \cup & [type : \{\text{“2b”}\},\ bal : Ballots,\ val : Values,\ acc : Acceptors]
\end{array}
$$

$$
\begin{array}{lll}
TypeOK & \triangleq & \land\ msgs \in \text{SUBSET}\ Messages \\
 & & \land\ maxVBal \in [Acceptors \rightarrow Ballots \cup \{-1\}] \\
 & & \land\ maxBal \in\ [Acceptors\ \rightarrow Ballots \cup \{-1\}] \\
 & & \land\ maxVal \in\ [Acceptors\ \rightarrow Values \cup \{None\}] \\
 & & \land\ \forall\, a \in Acceptors : maxBal[a] \geq maxVBal[a]
\end{array}
$$

NOTE: Messages defined as records.

Paxos — TLA$^+$ Specification (cont.)

Lamport (2001)

Phase $1a$: A leader selects a ballot number b and sends a $1a$ message with ballot b to a majority of acceptors. It can do this only if it has not already sent a $1a$ message for ballot b.

$$
\begin{aligned}
Phase1a(b) \triangleq\ & \land \neg\exists\, m \in msgs : (m.type = \text{“}\texttt{1a}\text{”}) \land (m.bal = b) \\
& \land Send([type \mapsto \text{“}\texttt{1a}\text{”},\ bal \mapsto b]) \\
& \land \text{UNCHANGED}\ \langle maxVBal,\ maxBal,\ maxVal \rangle
\end{aligned}
$$

Multiple $Send(_)$ in a single action would be a mistake.

$\neg\exists$ is only needed to make proof easier.

Paxos — TLA$^+$ Specification (cont.)

Lamport (2001)

Phase $1b$: If an acceptor receives a $1a$ message with ballot b greater than that of any $1a$ message to which it has already responded, then it responds to the request with a promise not to accept any more proposals for ballots numbered less than b and with the highest-numbered ballot (if any) for which it has voted for a value and the value it voted for in that ballot. That promise is made in a $1b$ message.

$$
\begin{aligned}
& Phase1b(a) \triangleq \\
& \quad \exists\, m \in msgs : \\
& \qquad \land\ m.type = \text{“1a”} \\
& \qquad \land\ m.bal > maxBal[a] \\
& \qquad \land\ Send([type \mapsto \text{“1b”},\ bal \mapsto m.bal,\ maxVBal \mapsto maxVBal[a], \\
& \qquad\qquad\qquad maxVal \mapsto maxVal[a],\ acc \mapsto a]) \\
& \qquad \land\ maxBal' = [maxBal \text{ EXCEPT } ![a] = m.bal] \\
& \qquad \land\ \text{UNCHANGED } \langle maxVBal,\ maxVal \rangle
\end{aligned}
$$

Each step is atomic — we update our initial state and send the message atomically.

That's not always easy to implement (consider node crashes, persistence vs network, . . .).

Paxos — TLA$^+$ Specification (cont.)

Lamport (2001)

Phase $2a$: If the leader receives a response to its $1b$ message (for ballot b) from a quorum of acceptors, then it sends a $2a$ message to all acceptors for a proposal in ballot b with a value v, where v is the value of the highest-numbered proposal among the responses, or is any value if the responses reported no proposals. The leader can send only one $2a$ message for any ballot.

$$
\begin{array}{l}
Phase2a(b) \triangleq \\
\quad \land \neg\exists\, m \in msgs : (m.type = \text{“2a”}) \land (m.bal = b) \\
\quad \land \exists\, v \in Values : \\
\qquad \land \exists\, Q \in Quorums : \\
\qquad\quad \exists\, S \in \text{SUBSET}\ \{m \in msgs : (m.type = \text{“1b”}) \land (m.bal = b)\} : \\
\qquad\qquad \land \forall\, a \in Q : \exists\, m \in S : m.acc \quad = a \\
\qquad\qquad \land \lor \forall\, m \in S : m.maxVBal = -1 \\
\qquad\qquad\quad \lor \exists\, c \in 0 \,..\, (b-1) : \\
\qquad\qquad\qquad \land \forall\, m \in S : m.maxVBal \leq c \\
\qquad\qquad\qquad \land \exists\, m \in S : \land\, m.maxVBal = c \\
\qquad\qquad\qquad\qquad\qquad\qquad \land\, m.maxVal = v \\
\qquad \land Send([type \mapsto \text{“2a”},\ bal \mapsto b,\ val \mapsto v]) \\
\quad \land \text{UNCHANGED}\ \langle maxBal,\ maxVBal,\ maxVal\rangle
\end{array}
$$

That's the most tricky part of the algorithm. $\neg\exists$ is important here.

Decision on messages directly, no typical reception actions.

Paxos — TLA$^+$ Specification (cont.)

Lamport (2001)

Phase $2b$: If an acceptor receives a $2a$ message for a ballot numbered b, it votes for the message's value in ballot b unless it has already responded to a $1a$ request for a ballot number greater than or equal to b.

$$
\begin{aligned}
& Phase2b(a) \triangleq \\
& \quad \exists\, m \in msgs : \\
& \qquad \land\ m.type = \text{“2a”} \\
& \qquad \land\ m.bal \geq maxBal[a] \\
& \qquad \land\ Send([type \mapsto \text{“2b”},\ bal \mapsto m.bal,\ val \mapsto m.val,\ acc \mapsto a]) \\
& \qquad \land\ maxVBal' = [maxVBal \text{ EXCEPT } ![a] = m.bal] \\
& \qquad \land\ maxBal' = [maxBal \text{ EXCEPT } ![a] = m.bal] \\
& \qquad \land\ maxVal' = [maxVal \text{ EXCEPT } ![a] = m.val]
\end{aligned}
$$

Message reception by learners is not modelled at all.

And there is no learners in the spec.

Paxos — TLA$^+$ Specification (cont.)

Lamport (2001)

How a value is chosen:

This spec does not contain any actions in which a value is explicitly chosen (or a chosen value learned). Wnat it means for a value to be chosen is defined by the operator *Chosen*, where $Chosen(v)$ means that v has been chosen. From this definition, it is obvious how a process learns that a value has been chosen from messages of type "2b".

$$VotedForIn(a, v, b) \triangleq \exists\, m \in msgs : \begin{aligned} &\wedge\, m.type = \text{“2b”} \\ &\wedge\, m.val = v \\ &\wedge\, m.bal = b \\ &\wedge\, m.acc = a \end{aligned}$$

$$ChosenIn(v, b) \triangleq \exists\, Q \in Quorums : \forall\, a \in Q : VotedForIn(a, v, b)$$

$$Chosen(v) \triangleq \exists\, b \in Ballots : ChosenIn(v, b)$$

The consistency condition that a consensus algorithm must satisfy is the invariance of the following state predicate *Consistency*.

$$Consistency \triangleq \forall\, v1, v2 \in Values : Chosen(v1) \wedge Chosen(v2) \Rightarrow (v1 = v2)$$

The final property is formulated based entirely on the sent messages.

Outline

Introduction

On Recursion

On Composition

Refinements
- Example: Connection Locks
- Example: Paxos

Discussion and Follow-up

References

Discussion

Main points to consider:

- Select appropriate abstraction level for the spec.
- Sometimes it is worth to select several abstraction levels and then show refinement relation between them.
- Consider, when operational and when denotational semantics should be chosen for a spec.
- Refinement can be considered as a special case of a temporal property – the behaviour should match the pattern defined by the abstract spec.
- Spec is not a program. You don't need the entire sub-module to be included to use its properties.

Follow-up

Post-readings:

- Specifying Systems by Lamport (2002)
  Chapter 4: A FIFO;
  Chapter 7: Writing Specification: Some Advice;
  Chapter 10: Composing Specifications.

Related recent research:

- Verifying Hyperproperties with TLA by Lamport and Schneider (2021)
- Prophecy Made Simple by Lamport and Merz (2020)
- Auxiliary Variables in TLA+ by Lamport and Merz (2017)

Outline

Introduction

On Recursion

On Composition

Refinements
- Example: Connection Locks
- Example: Paxos

Discussion and Follow-up

References

References

Lamport, L. (1998). The part-time parliament. *ACM Trans. Comput. Syst.*, 16(2):133–169.

Lamport, L. (2001). Paxos made simple. *ACM SIGACT News (Distributed Computing Column) 32, 4 (Whole Number 121, December 2001)*, pages 51–58.

Lamport, L. (2002). *Specifying Systems: The TLA+ Language and Tools for Hardware and Software Engineers*. Addison-Wesley Longman Publishing Co., Inc., USA.

Lamport, L. (2019). Hiding, refinement, and auxiliary variables.

Lamport, L. and Merz, S. (2017). Auxiliary variables in tla+. Unpublished.

Lamport, L. and Merz, S. (2020). Prophecy made simple. Submitted for Publication.

Lamport, L. and Schneider, F. B. (2021). Verifying hyperproperties with tla. To appear in the Proceedings of the 34th IEEE Computer Security Foundations Symposium, CSF 2021, Dubrovnik, Croatia, June 21-25.